LabLink

Lab & Equipment Reservation System

Milestone 2: Design & Architecture

CS450 Software Engineering · Campbellsville University
Professor Sean Perryman

Team: Weston Mattingly · Isaiah Bain · Dawson Brown · PJ

October 18, 2026

Repository: https://github.com/westonm05/Reservation-Project

Contents

1. Architecture Document
2. UML Models
3. Database Design
4. Architecture Decision Records (ADR-0001 – 0004)
5. Project Plan

LabLink Architecture Document

CS450 Software Engineering, Milestone 2 — Design & Architecture Team: Weston Mattingly, Isaiah Bain, Dawson Brown, PJ

1. System overview

LabLink is a web application that lets students and faculty reserve lab rooms and equipment, and lets lab managers maintain the inventory and approve reservation requests. Users interact through a browser; all pages are rendered on the server and data is stored in a relational database.

Actors

Actor	Can do
Student	Register, log in, browse labs and equipment, request reservations (need approval), view and cancel own reservations
Faculty	Everything a student can; reservations are approved automatically
Lab Manager	Everything above, plus create/edit/delete labs and equipment and approve or reject pending requests

Core business rules

1. A reservation is for exactly one lab **or** one piece of equipment.
2. A reservation must start in the future and end after it starts.
3. A lab or piece of equipment cannot be double-booked: a new request is refused if it overlaps any PENDING or APPROVED reservation for the same resource.
4. Inactive labs and equipment that is in MAINTENANCE or RETIRED cannot be reserved.
5. Only the person who made a reservation can cancel it; only a lab manager can approve or reject it.

2. Architectural pattern: Layered architecture with MVC

Selection

LabLink is a **single deployable application (a "modular monolith") organized into four layers**, with the web layer following the **Model–View–Controller (MVC)** pattern:

Layer	Responsibility	LabLink packages
Presentation (View + Controller)	Receive HTTP requests, validate form input, choose a page to render	`web`, `web.form`, `templates/`
Business (Service)	Enforce the business rules above; coordinate transactions	`service`
Persistence (Repository)	Read and write entities; no business logic	`repository`
Domain (Model)	Entities that map to database tables	`model`
Cross-cutting	Authentication and authorization applied to every request	`security`, `config`

Each layer may only call the layer directly below it: controllers call services, services call repositories, repositories talk to the database. Controllers never touch repositories for reservations, and templates never contain business logic.

Justification

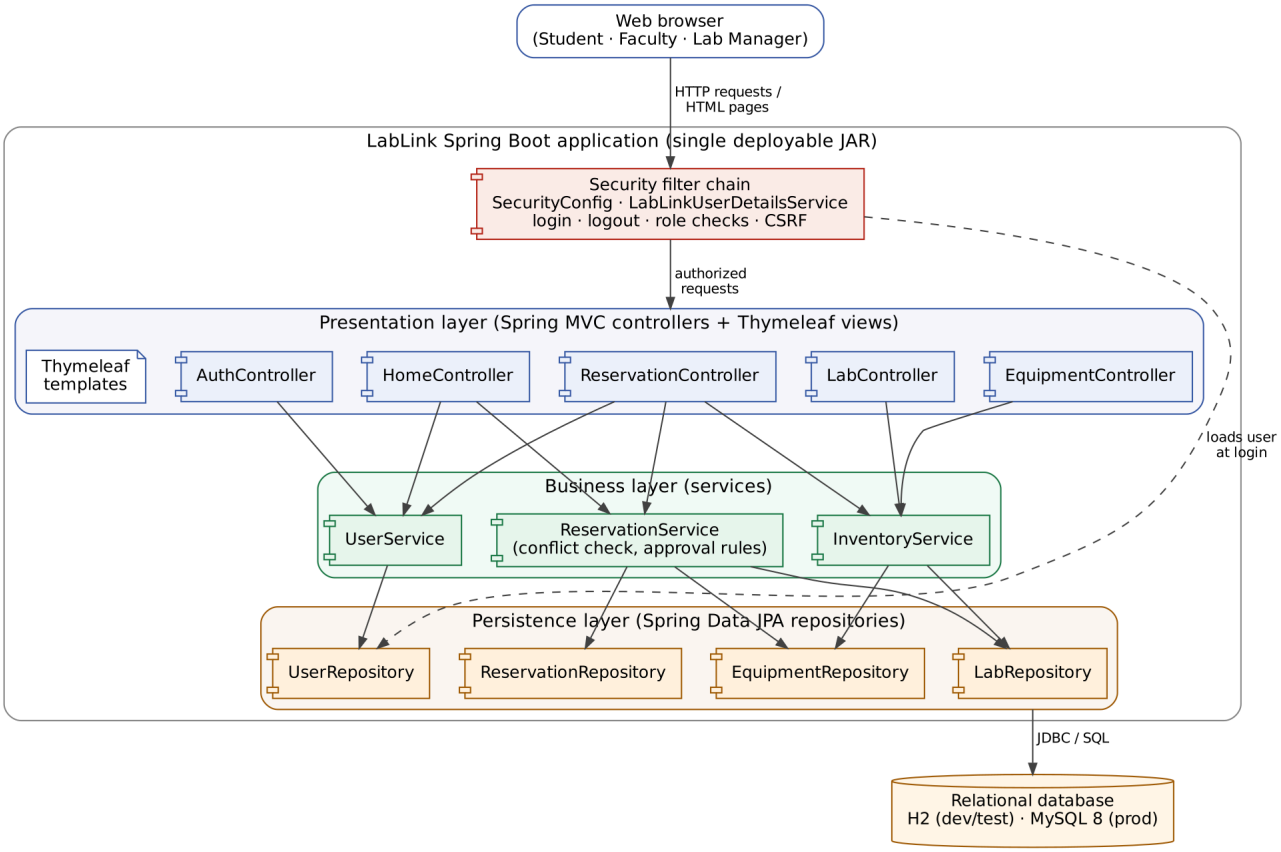
- **Fits the problem size.** LabLink has four entities and three roles. A layered monolith gives clear structure without the overhead of distributed services.
- **Fits the team.** Every team member is new to building applications. Layers give each person an obvious place to put code and map directly onto the team roles: authentication (`security`, `AuthController`), reservations (`ReservationService`), front end (`templates/`), and database/inventory (`repository`, `InventoryService`, `schema.sql`).
- **Fits the framework.** Spring Boot is built around this pattern (`@Controller`, `@Service`, `@Repository`), so we follow the framework instead of fighting it.
- **Testability.** Business rules live in one class (`ReservationService`) and can be tested against a real in-memory database without a browser.
- **One deployment.** A single JAR is easy to run locally and deploy for the final demo.

Alternatives considered

Alternative	Why not chosen
Microservices (separate auth, inventory, reservation services)	Network calls, multiple deployments and distributed data are far more work than the course timeline allows and add no value at this scale.
REST API + single-page JavaScript front end (React)	Requires two codebases and two skill sets; server-rendered Thymeleaf lets the whole team work in Java. A REST endpoint can still be added later for the API/advanced-feature requirement.
Unstructured “controllers do everything”	Fast at first, but business rules end up duplicated across pages and are hard to test.

See ADR-0001 for the recorded decision.

3. Component diagram



Source: diagrams/component.dot (Graphviz). Every controller also renders a Thymeleaf template from `src/main/resources/templates`.

Which component calls which

Controller	Services it uses
AuthController	UserService
HomeController	UserService, ReservationService
LabController, EquipmentController	InventoryService
ReservationController	ReservationService, InventoryService, UserService

Service	Repositories it uses
UserService	UserRepository (also used by LabLinkUserDetailsService at login)
InventoryService	LabRepository, EquipmentRepository
ReservationService	ReservationRepository, LabRepository, EquipmentRepository

How a request flows: the browser sends a request → the security filter chain checks that the user is logged in and has the right role → a controller validates the form and calls a service → the service applies the business rules and uses repositories to read/write data inside a transaction → the controller picks a Thymeleaf template, which renders HTML back to the browser.

4. Technology stack

Concern	Technology	Version	Why
Language	Java	17 (LTS)	Taught in our courses; every team member already knows it
Application framework	Spring Boot	3.5	Industry standard; auto-configuration removes most setup work
Web / MVC	Spring MVC	(via Boot)	Controllers, form binding, validation
Views	Thymeleaf + Spring Security extras	(via Boot)	Server-rendered HTML in plain Java; show/hide by role
Security	Spring Security, BCrypt	(via Boot)	Form login, sessions, role-based access, CSRF protection, password hashing
Persistence	Spring Data JPA / Hibernate	(via Boot)	Repositories generate queries from method names
Validation	Jakarta Bean Validation	(via Boot)	`@NotBlank`, `@Email`, etc. on forms
Database (dev/test)	H2, in memory, MySQL mode	2.x	Zero install; fresh data on every run
Database (production)	MySQL	8.x	Free, widely hosted, supports CHECK constraints
Schema management	Hand-written DDL (`db/schema.sql`)	—	The DDL in the design document is exactly what runs (ADR-0003)
Build	Maven	3.9	Standard for Spring; dependency management
Testing	JUnit 5, Spring Boot Test, MockMvc, AssertJ	(via Boot)	Service tests against H2; full HTTP tests through security
CI	GitHub Actions	—	Builds and runs all tests on every push and pull request
Version control	Git + GitHub, feature-branch workflow	—	Reviewed pull requests into `main`

Concern	Technology	Version	Why
Hosting (final demo)	Cloud host running the JAR + managed MySQL	TBD in Milestone 3	—

5. Security design

- Passwords are hashed with BCrypt; plain-text passwords are never stored or logged.
- Self-registration always creates a **STUDENT** account. Faculty and lab manager accounts are created by an administrator (seeded in development), so users cannot grant themselves privileges.
- URL rules in `SecurityConfig` restrict inventory changes and request review to `ROLE_LAB_MANAGER`. The service layer re-checks the role for approve/reject and ownership for cancel, so a rule cannot be bypassed by calling a URL directly.
- All state-changing actions are HTTP POST requests protected by Spring Security's CSRF token, which Thymeleaf adds to every form automatically.
- Database credentials for production come from environment variables, not from the repository.

6. Prototype scope (Milestone 2)

The prototype in the repository demonstrates every layer end to end:

- Registration, login and logout with three roles
- CRUD for labs and equipment (lab manager)
- Reservation requests with double-booking prevention, faculty auto-approval, manager approve/reject, and owner cancellation
- DDL-driven schema on H2 with demo data
- 18 automated tests (business rules and HTTP/security) run by GitHub Actions

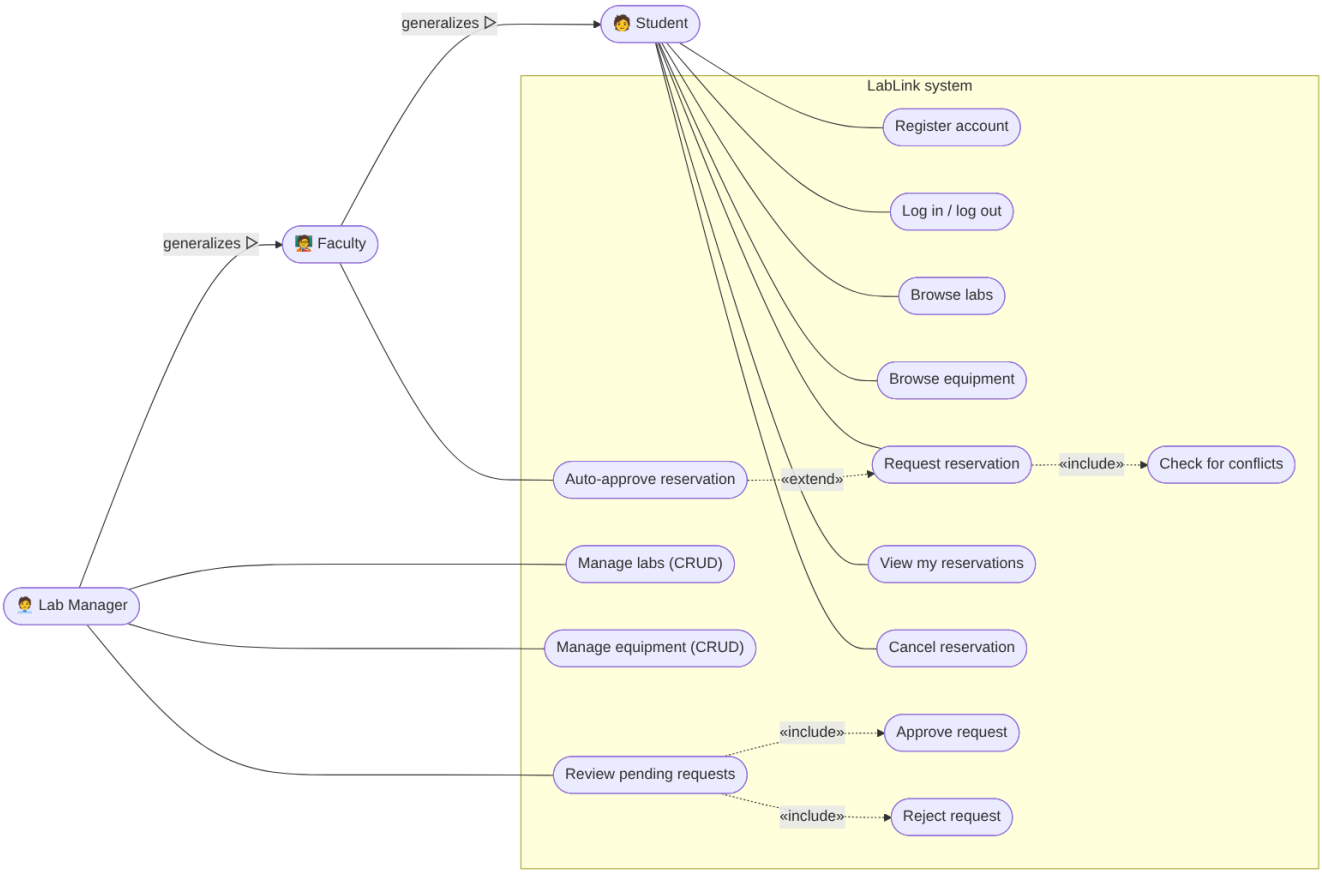
Planned for later milestones: production MySQL deployment, reservation calendar view, email notifications or a public availability API (advanced feature / API integration requirement), and UI polish.

LabLink UML Models

All diagrams describe the code in the repository as of Milestone 2.

1. Use case diagram

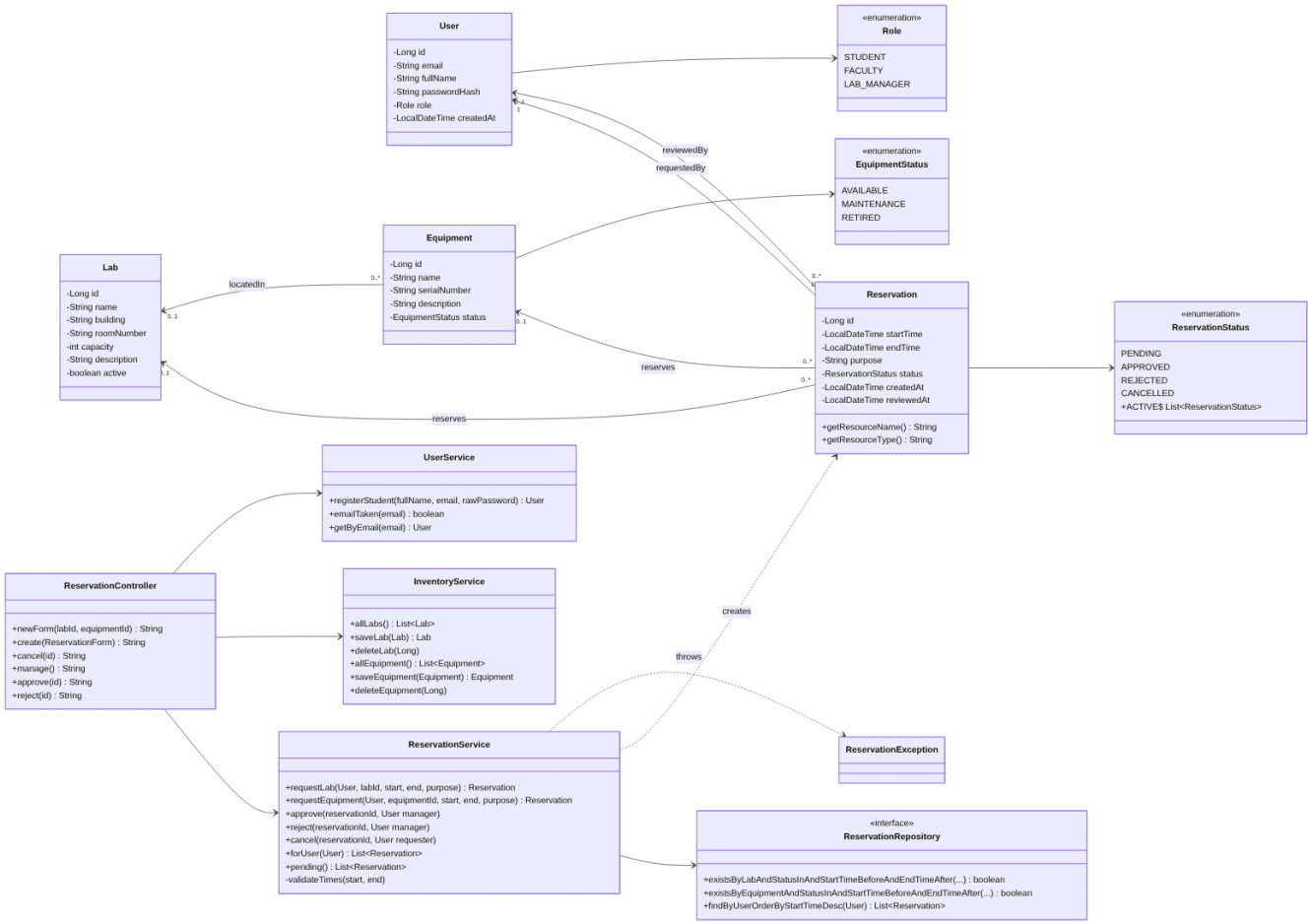
Actors are on the left; use cases (ovals) sit inside the LabLink system boundary. Faculty and Lab Manager inherit every use case of the actor they generalize.



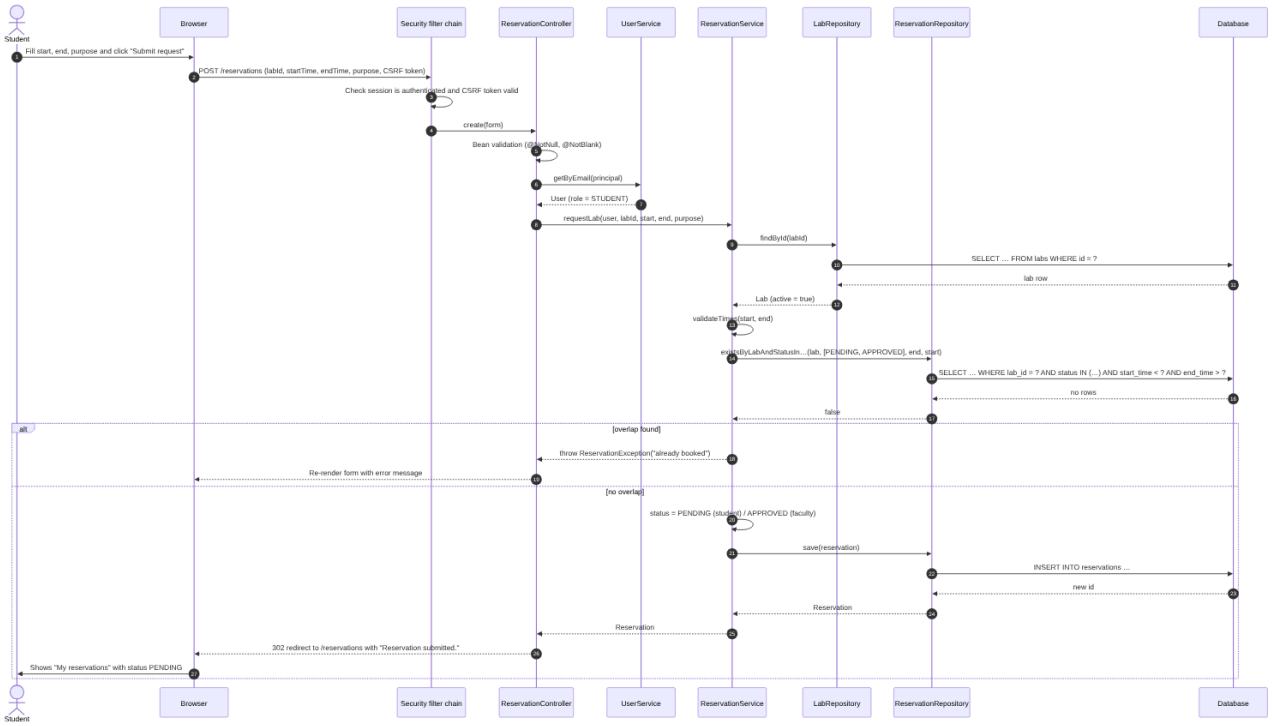
Use case	Actor	Summary
Register account	Student	Create a STUDENT account with name, email and password
Log in / log out	All	Authenticate with email and password
Browse labs / equipment	All	List rooms and items with location and status
Request reservation	All	Pick a resource and time window; the system checks for conflicts
Auto-approve	Faculty, Lab Manager	Requests from these roles skip the approval queue
View / cancel my reservations	All	See own reservations; cancel ones still pending or approved
Manage labs / equipment	Lab Manager	Create, edit, delete inventory
Review / approve / reject	Lab Manager	Decide on students' pending requests

2. Class diagram

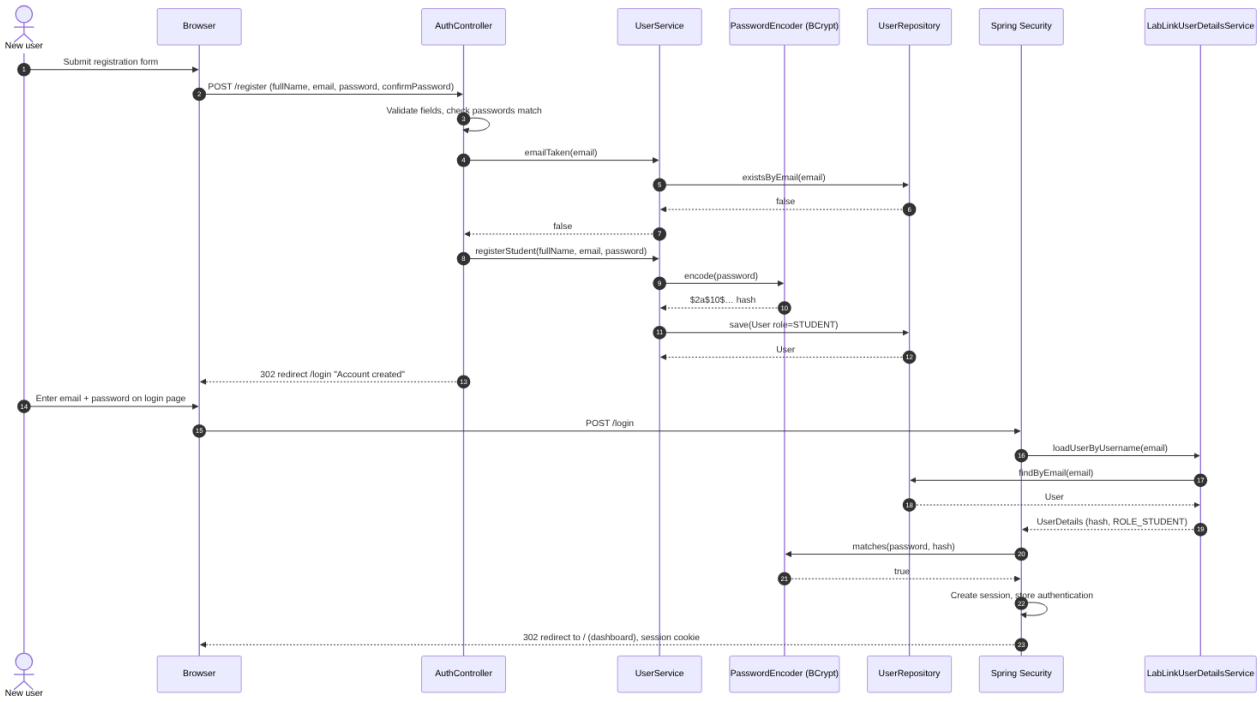
Domain entities, the services that hold business rules, and how controllers and repositories connect to them. (Getters/setters omitted.)



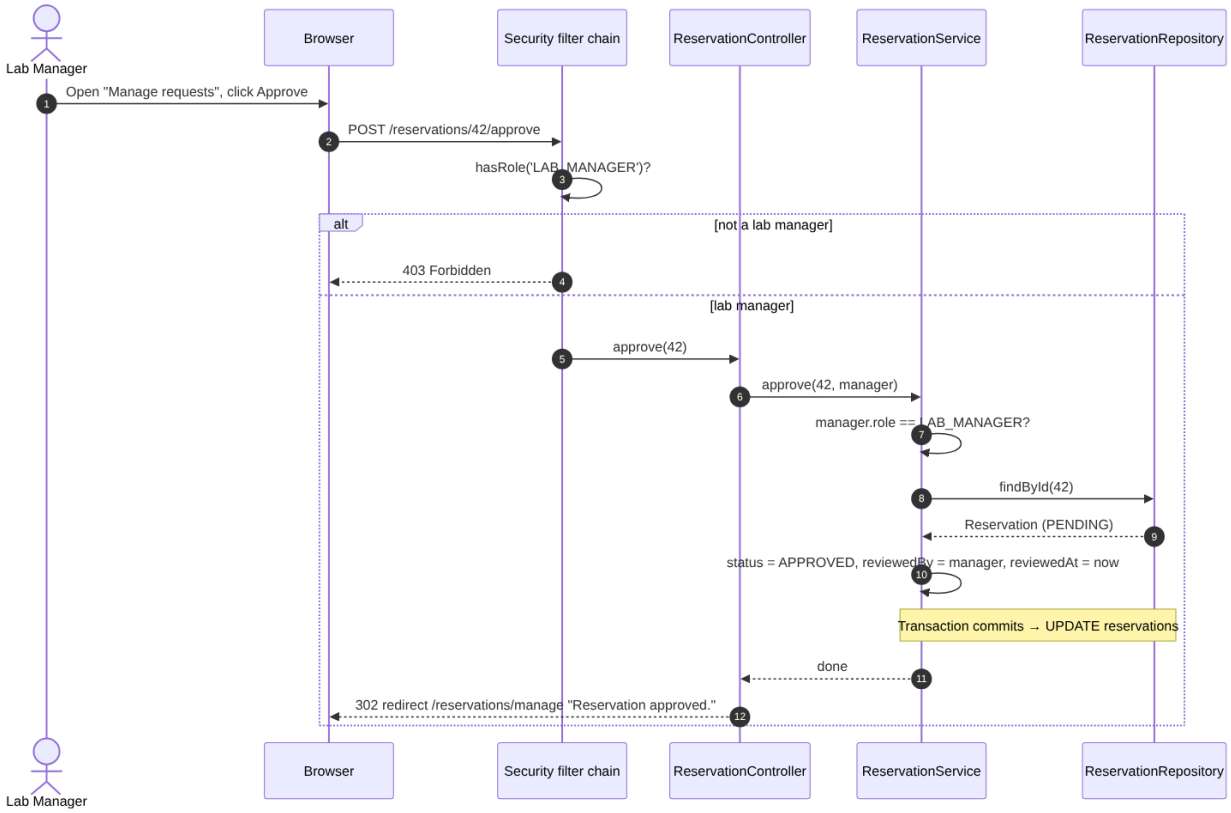
3. Sequence diagram — Request a lab reservation (with conflict check)



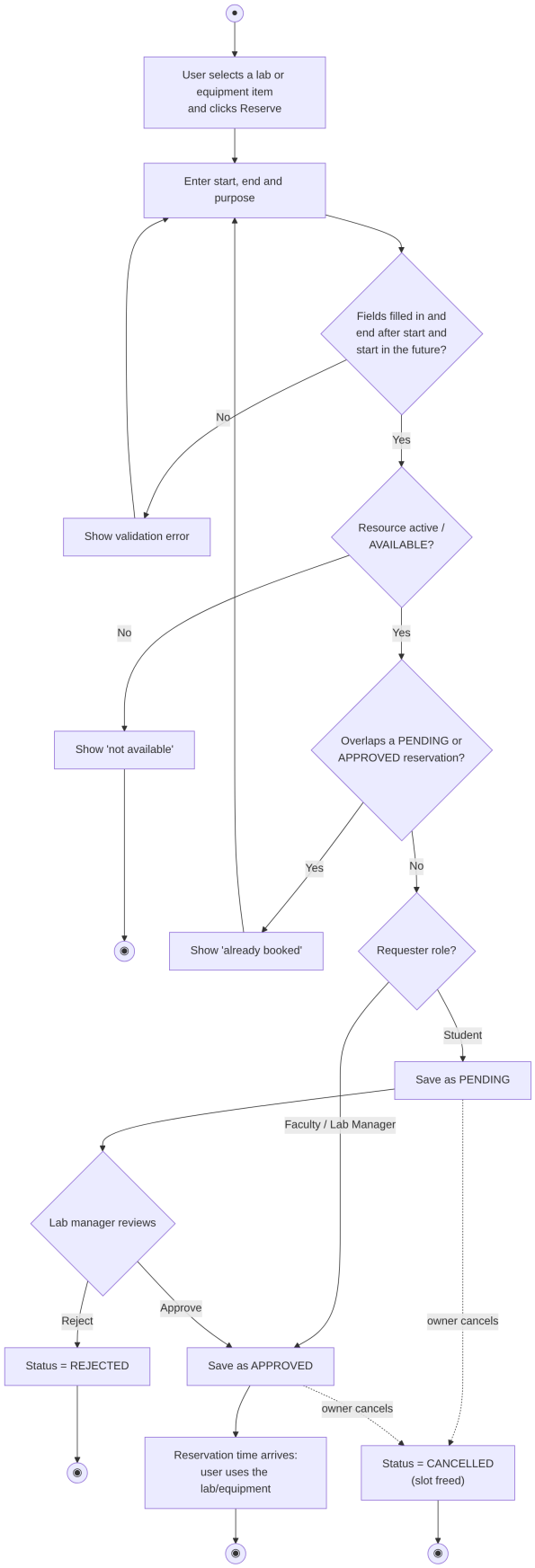
4. Sequence diagram — Register and log in



5. Sequence diagram — Lab manager approves a request

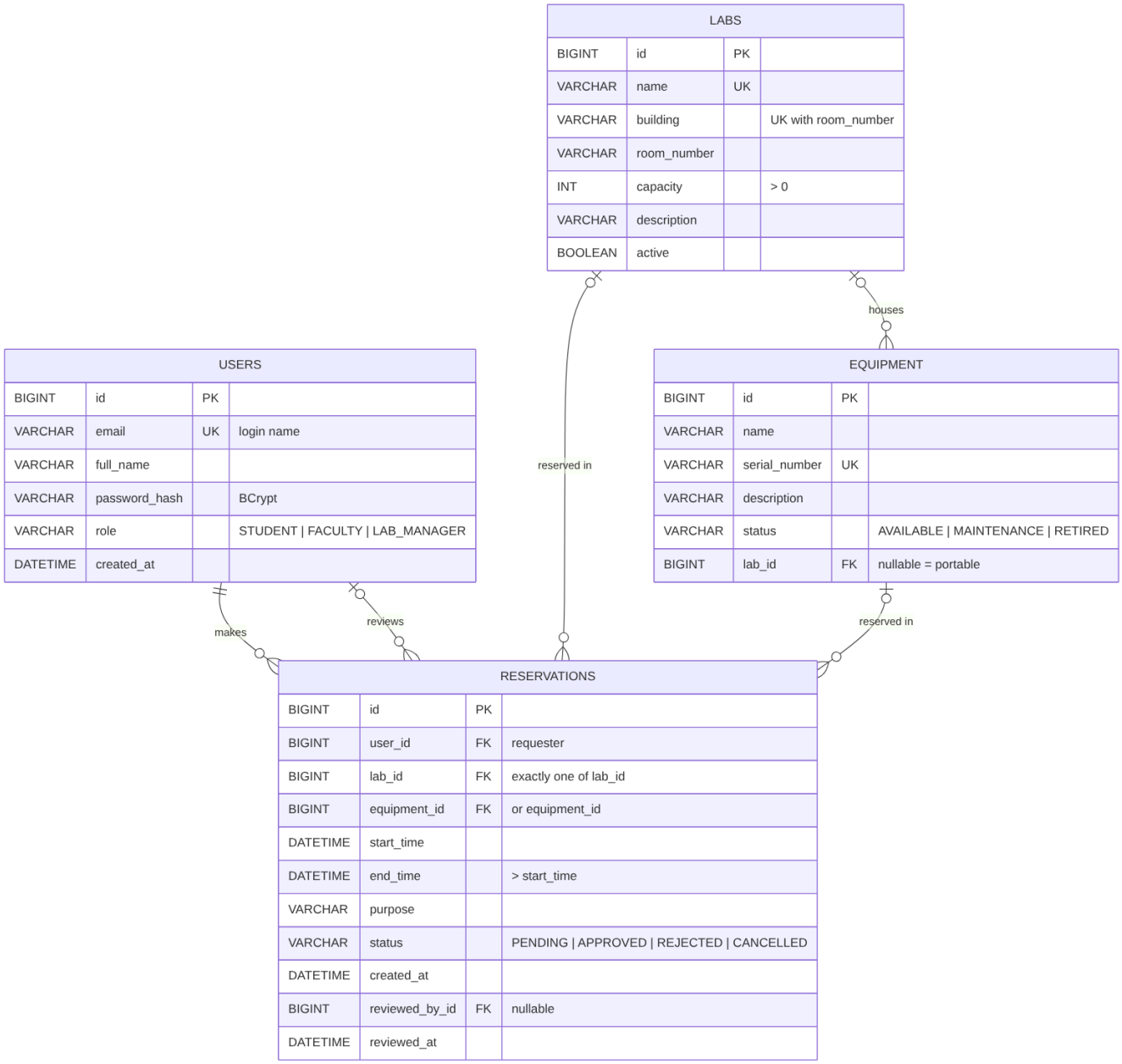


6. Activity diagram — Reservation life cycle


User selects a lab or equipment item and clicks Reserve
Enter start, end and purpose
Fields filled in and end after start and start in the future?
No
Yes
Show validation error
Resource active / AVAILABLE?
No
Yes
Show 'not available'
Overlaps a PENDING or APPROVED reservation?
Yes
No
Show 'already booked'
Requester role?
Student
Save as PENDING
Faculty / Lab Manager
Lab manager reviews
Reject
Approve
owner cancels
Status = REJECTED
Save as APPROVED
owner cancels
Reservation time arrives: user uses the lab/equipment
Status = CANCELLED (slot freed)

LabLink Database Design

1. ER diagram



Relationships

Relationship	Cardinality	Notes
User makes Reservation	1 to 0..*	Every reservation has exactly one requester
User reviews Reservation	0..1 to 0..*	Set when a lab manager approves/rejects
Lab houses Equipment	0..1 to 0..*	Equipment with no lab is portable
Lab / Equipment reserved in Reservation	0..1 to 0..*	A reservation targets exactly one lab **or** one equipment item (CHECK constraint)

2. Normalized schema (3NF)

Relations

- **users**(id, email, full_name, password_hash, role, created_at)
- **labs**(id, name, building, room_number, capacity, description, active)
- **equipment**(id, name, serial_number, description, status, *lab_id* → labs)
- **reservations**(id, *user_id* → users, *lab_id* → labs, *equipment_id* → equipment, start_time, end_time, purpose, status, created_at, *reviewed_by_id* → users, reviewed_at)

Candidate keys: users.email; labs.name; labs.(building, room_number); equipment.serial_number.

Functional dependencies

Relation	Dependencies
users	id → email, full_name, password_hash, role, created_at; email → id (candidate key)
labs	id → all attributes; name → id; (building, room_number) → id
equipment	id → name, serial_number, description, status, lab_id; serial_number → id
reservations	id → all attributes

Normal form check

First Normal Form (1NF): every column holds one atomic value; there are no repeating groups or lists (e.g. a lab's equipment is not stored as a comma-separated column — it is a separate table with a foreign key). Every table has a primary key. ✔

Second Normal Form (2NF): all primary keys are single-column surrogate keys (`id`), so no attribute can depend on only part of a key. ✔

Third Normal Form (3NF): no non-key attribute depends on another non-key attribute. Decisions that keep it that way:

- `reservations` stores only `user_id`, never the requester's name or email. Storing `user_name` there would create the transitive dependency `id → user_id → user_name` and would go stale if the user changed their name.
- `reservations` stores `lab_id` / `equipment_id`, not the lab's building or room; those depend on the lab, not the reservation.
- `equipment` stores `lab_id`, not the lab's name or building.
- The only determinants on the left of a dependency are candidate keys (`id`, `email`, `name`, `(building, room_number)`, `serial_number`), so the schema also satisfies **Boyce–Codd Normal Form**. ✔

Example of what we avoided (unnormalized):

reservation_id	student_name	student_email	lab_name	building	room	start	end
1	Ana Ruiz	ana@...	Robotics Lab	Science	118	10/20 9:00	10/20 11:00
2	Ana Ruiz	ana@...	Robotics Lab	Science	118	10/21 9:00	10/21 10:00

Here `student_name` depends on `student_email` and `building` / `room` depend on `lab_name` (transitive dependencies), so changing Ana's name or moving the lab means updating many rows (update anomaly), and a lab with no reservations can't be recorded at all (insertion anomaly). Splitting into `users`, `labs` and `reservations` removes these anomalies.

Design notes

- **Enumerated values** (role, statuses) are stored as text with CHECK constraints rather than lookup tables. The value sets are fixed in code and small; text values keep rows readable. This does not violate 3NF (the status is a fact about the row, not determined by another non-key column).
- **One reservations table with two nullable FKs** instead of separate `lab_reservations` and `equipment_reservations` tables keeps "my reservations" and the approval queue to one simple query. The CHECK constraint guarantees exactly one target.
- **Indexes:** MySQL InnoDB and H2 automatically index foreign-key columns, which serves the double-booking lookup (`lab_id`, `equipment_id`) and "my reservations" (`user_id`). Unique constraints create indexes on email, lab name/location and serial number.

3. DDL script

The application runs this exact script at startup: `src/main/resources/db/schema.sql`. It is compatible with MySQL 8 and H2 (MySQL mode).

```
        CREATE TABLE IF NOT EXISTS users (
            id            BIGINT       NOT NULL AUTO_INCREMENT,
            email         VARCHAR(255) NOT NULL,
            full_name     VARCHAR(100) NOT NULL,
            password_hash VARCHAR(100) NOT NULL,
            role          VARCHAR(20)  NOT NULL,
            created_at    DATETIME     NOT NULL,
            CONSTRAINT pk_users PRIMARY KEY (id),
            CONSTRAINT uq_users_email UNIQUE (email),
            CONSTRAINT chk_users_role CHECK (role IN ('STUDENT', 'FACULTY',
'LAB_MANAGER'))
        );

        CREATE TABLE IF NOT EXISTS labs (
            id          BIGINT       NOT NULL AUTO_INCREMENT,
            name        VARCHAR(100) NOT NULL,
            building    VARCHAR(100) NOT NULL,
            room_number VARCHAR(20)  NOT NULL,
            capacity    INT          NOT NULL,
            description VARCHAR(500),
            active      BOOLEAN      NOT NULL DEFAULT TRUE,
            CONSTRAINT pk_labs PRIMARY KEY (id),
            CONSTRAINT uq_labs_name UNIQUE (name),
            CONSTRAINT uq_labs_location UNIQUE (building, room_number),
            CONSTRAINT chk_labs_capacity CHECK (capacity > 0)
        );

        CREATE TABLE IF NOT EXISTS equipment (
            id            BIGINT       NOT NULL AUTO_INCREMENT,
            name          VARCHAR(100) NOT NULL,
            serial_number VARCHAR(50)  NOT NULL,
            description   VARCHAR(500),
            status        VARCHAR(20)  NOT NULL,
            lab_id        BIGINT,
            CONSTRAINT pk_equipment PRIMARY KEY (id),
            CONSTRAINT uq_equipment_serial UNIQUE (serial_number),
            CONSTRAINT fk_equipment_lab FOREIGN KEY (lab_id) REFERENCES labs (id),
            CONSTRAINT chk_equipment_status CHECK (status IN ('AVAILABLE', 'MAINTENANCE',
'RETIRED'))
        );

        CREATE TABLE IF NOT EXISTS reservations (
            id             BIGINT       NOT NULL AUTO_INCREMENT,
            user_id        BIGINT       NOT NULL,
            lab_id         BIGINT,
            equipment_id   BIGINT,
            start_time     DATETIME     NOT NULL,
            end_time       DATETIME     NOT NULL,
            purpose        VARCHAR(255) NOT NULL,
            status         VARCHAR(20)  NOT NULL,
            created_at     DATETIME     NOT NULL,
            reviewed_by_id BIGINT,
```

```
            reviewed_at    DATETIME,
            CONSTRAINT pk_reservations PRIMARY KEY (id),
            CONSTRAINT fk_reservations_user FOREIGN KEY (user_id) REFERENCES users (id),
            CONSTRAINT fk_reservations_lab FOREIGN KEY (lab_id) REFERENCES labs (id),
            CONSTRAINT fk_reservations_equipment FOREIGN KEY (equipment_id) REFERENCES
equipment (id),
            CONSTRAINT fk_reservations_reviewer FOREIGN KEY (reviewed_by_id) REFERENCES
users (id),
            CONSTRAINT chk_reservations_times CHECK (end_time > start_time),
            CONSTRAINT chk_reservations_status CHECK (status IN ('PENDING', 'APPROVED',
'REJECTED', 'CANCELLED')),
            CONSTRAINT chk_reservations_target CHECK (
                (lab_id IS NOT NULL AND equipment_id IS NULL) OR
                (lab_id IS NULL AND equipment_id IS NOT NULL))
        );
```

Sample queries the application issues

```
        -- Double-booking check for a lab (ADR-0004)
        SELECT COUNT(*) > 0
        FROM reservations
        WHERE lab_id = ?
          AND status IN ('PENDING', 'APPROVED')
          AND start_time < ?   -- requested end
          AND end_time   > ?;  -- requested start

        -- Approval queue
        SELECT r.*, u.full_name
        FROM reservations r JOIN users u ON u.id = r.user_id
        WHERE r.status = 'PENDING'
        ORDER BY r.start_time;
```

ADR-0001: Use a layered MVC monolith

- **Status:** Accepted
- **Date:** 2026-10-18
- **Deciders:** LabLink team

Context

LabLink needs a front end, a database, authentication with roles, and CRUD on several entities, delivered by a four-person student team across one semester. Team members are new to building applications. The application has a small domain (users, labs, equipment, reservations) and modest load (one campus).

Decision

Build LabLink as a single Spring Boot application organized into presentation (controllers + Thymeleaf views), business (services), persistence (Spring Data repositories) and domain (JPA entities) layers. Each layer only calls the layer below it. Business rules live in services, not in controllers or templates.

Consequences

Positive - One codebase, one build, one deployment; simple to run locally (`mvn spring-boot:run`). - Clear place for every kind of code; layers map onto team roles. - Business rules can be unit/integration tested without a browser. - Matches Spring's conventions and most tutorials the team will find.

Negative - The whole application scales as one unit (acceptable at campus scale). - Discipline is needed to keep controllers thin; code review checks for business logic in controllers.

Alternatives rejected: microservices (operational overhead far exceeds benefit at this size); separate REST API + React SPA (two codebases and skill sets; can add a REST endpoint later if needed).

ADR-0002: Server-side rendering with Thymeleaf and session-based Spring Security

- **Status:** Accepted
- **Date:** 2026-10-18
- **Deciders:** LabLink team

Context

The project requires a front end and authentication with role-based access control. Options were (a) server-rendered HTML with Thymeleaf and form login backed by an HTTP session, or (b) a JavaScript single-page app calling a REST API secured with JSON Web Tokens (JWT).

Decision

Render pages on the server with Thymeleaf and use Spring Security form login with server-side sessions. Roles (`STUDENT` , `FACULTY` , `LAB_MANAGER`) are stored on the user and mapped to Spring authorities (`ROLE_…`). Access is enforced twice: URL rules in `SecurityConfig` , and role/ownership checks in the service layer. Passwords are hashed with BCrypt. Only students can self-register.

Consequences

Positive - Entire application is Java; no separate front-end build or framework to learn. - CSRF protection, session handling, login/logout pages and password hashing come built in. - `sec:authorize` in templates hides buttons a user is not allowed to use.

Negative - Pages reload on every action (less “app-like” than an SPA). - A future mobile client would need a REST API with token auth added alongside.

Alternatives rejected: JWT + SPA — more moving parts (token storage, refresh, CORS) and a second language/toolchain for the team.

ADR-0003: H2 for development, MySQL for production, schema from hand-written DDL

- **Status:** Accepted
- **Date:** 2026-10-18
- **Deciders:** LabLink team

Context

Every developer must be able to run the application immediately, CI must run tests without a database server, and the final demo needs a persistent production database. The course also requires DDL scripts as a design deliverable. Hibernate can generate tables automatically (`ddl-auto=update`), but then the real schema can drift from the documented one.

Decision

- Use **H2 in memory, in MySQL compatibility mode,** as the default database for development and tests. Data is recreated on every start and filled with demo accounts by `DevDataSeeder`.
- Use **MySQL 8** in production via the `mysql` Spring profile, with credentials from environment variables.
- Create tables from one hand-written script, **`src/main/resources/db/schema.sql`**, run by Spring at startup (`spring.sql.init.mode=always`). Hibernate schema generation is turned off (`ddl-auto=none`). The script uses `CREATE TABLE IF NOT EXISTS` so it is safe to run repeatedly, and only SQL that both H2 and MySQL accept.

Consequences

Positive - Zero setup for teammates and CI; tests run against the same DDL as production. - The DDL in the design document is literally what the application runs, including CHECK, UNIQUE and FOREIGN KEY constraints. - Constraints protect data even if application code has a bug.

Negative - Schema changes must be made by hand in `schema.sql` and in the entity classes together. - `IF NOT EXISTS` does not alter existing tables; once production holds real data, we will adopt a migration tool (Flyway) — revisit in Milestone 3. - H2's MySQL mode is close to, but not identical to, MySQL; we will run the app against a real MySQL instance before the final demo.

ADR-0004: Enforce double-booking prevention in the service layer with an overlap query

- **Status:** Accepted
- **Date:** 2026-10-18
- **Deciders:** LabLink team

Context

The most important rule in LabLink is that a lab or piece of equipment cannot be reserved by two people at the same time. Options: check in the controller, check in the service, or rely only on a database constraint. Standard SQL cannot express “no overlapping time ranges” as a simple constraint in MySQL.

Decision

`ReservationService` checks for conflicts before saving, inside the same transaction, using a repository query that implements the interval-overlap test:

```
existing.start_time < requested.end_time  AND  existing.end_time > requested.start_time
```

restricted to the same resource and to statuses that still hold the slot (`PENDING`, `APPROVED`). Back-to-back bookings (one ends at 10:00, the next starts at 10:00) are allowed. Pending requests hold the slot so two students cannot both be waiting for the same time.

Consequences

Positive - One authoritative place for the rule, covered by automated tests (`ReservationServiceTest`). - Clear error message shown to the user (“Robotics Lab is already booked during that time”). - Foreign-key indexes on `lab_id` / `equipment_id` keep the lookup fast.

Negative - Two requests submitted at exactly the same instant could both pass the check (race condition). Acceptable for campus-scale traffic in the prototype; if needed later, lock the resource row (`SELECT … FOR UPDATE`) before checking.

LabLink Project Plan

Last updated: October 18, 2026 (end of Milestone 2)

1. Team and roles

Member	Role	Owns
Weston Mattingly	Project Coordinator, Authentication	Planning, communication, registration/login/security
Isaiah Bain	Reservations, Code Review Lead	Reservation workflow, pull-request reviews
Dawson Brown	Front End, UI/UX Lead	Thymeleaf templates, styling, usability
PJ	Database, Inventory, QA/DevOps	Schema, lab/equipment CRUD, tests, CI, deployment

2. Milestones

Milestone	Focus	Due	Status
1	Proposal, project charter, repository setup	Sept 2026	✅ Complete
2	Design & architecture + working prototype	Oct 18, 2026	✅ Complete (this submission)
3	Core features complete, MySQL integration, API/advanced feature	Target: mid-Nov 2026	⏳ Not started
4	Polish, testing, deployment, final demo	Target: early Dec 2026	⏳ Not started

(Milestone 3 and 4 dates are targets; adjust to the syllabus dates.)

3. Milestone 2 progress

Deliverable	Location	Status	Completed by
Architecture document (pattern, component diagram, tech stack)	`docs/architecture.md`	✅ Done	Weston
UML: use case, class, 3 sequence, activity	`docs/uml.md`	✅ Done	Weston
ER diagram, 3NF schema, DDL	`docs/database-design.md` , `src/main/resources/db/schema.sql`	✅ Done	Weston
ADRs (4)	`docs/adr/`	✅ Done	Weston
Working prototype	`src/`	✅ Done	Weston
CI pipeline (GitHub Actions)	`.github/workflows/ci.yml`	✅ Done	Weston
Updated project plan	this file	✅ Done	Weston

Prototype features working now: registration and login with three roles; lab and equipment CRUD for lab managers; reservation requests with double-booking prevention; faculty auto-approval; lab-manager approve/reject; owner cancellation; 18 automated tests.

4. Milestone 3 work breakdown

#	Task	Owner	Estimate	Depends on
3.1	Run app against real MySQL; add Flyway migrations (ADR-0003 follow-up)	PJ	4 h	—
3.2	Reservation calendar/availability view per lab	Isaiah	6 h	—
3.3	Edit/reschedule a pending reservation	Isaiah	3 h	—
3.4	Admin page to create faculty/lab-manager accounts and change roles	Weston	4 h	—
3.5	Password change and account page	Weston	3 h	—
3.6	Advanced feature: email notification on approve/reject **or** public REST availability API (team to choose)	Weston + Isaiah	8 h	3.2
3.7	UI redesign: layout, navigation, mobile-friendly tables, form feedback	Dawson	8 h	—
3.8	Search/filter labs and equipment	Dawson	3 h	—
3.9	Tests for inventory CRUD and new features; keep CI green	PJ	4 h	ongoing
3.10	Update docs/diagrams for new features	All	2 h	all

5. Milestone 4 outline

- Deploy to a cloud host with managed MySQL; environment-variable configuration
- End-to-end test pass with demo script for each role

- Bug fixing, accessibility check, final documentation and user guide
- Final presentation and demo

6. Process

- **Branching:** `feature/<name>` branches → pull request → at least one review (Isaiah, code review lead) → merge to `main`. No direct commits to `main`.
- **CI:** every push and pull request runs `mvn verify`; a failing build blocks merging.
- **Communication:** weekly check-in coordinated by Weston; tasks tracked as GitHub Issues assigned to owners above.
- **Definition of done:** code merged through a reviewed PR, tests added and passing, docs updated if behavior changed.

7. Risks

Risk	Likelihood	Impact	Mitigation
Uneven contribution across team members	High (occurred in M2)	High	Assign M3 tasks as GitHub Issues with owners and dates; review progress at weekly check-ins; report through peer evaluations
Team new to Spring Boot	High	Medium	Commented prototype code as a reference; pair on first feature each
H2 vs MySQL differences	Medium	Medium	Test against MySQL early in M3 (task 3.1)
Deployment issues close to the final demo	Medium	High	Deploy a first version during M3, not M4
Double-booking race condition	Low	Low	Acceptable at campus scale; row locking documented in ADR-0004